Highlights

Position Method: A Linear-Time Generation Algorithm for Permutations

Yusheng Hu

- A novel position-driven paradigm for permutation generation is proposed.
- Achieves $O(n)$ complexity for both Rank and Unrank operations.
- Demonstrates 5–10x speedup compared to the classical Heap's algorithm.
- Enables high parallelism with near-linear speedup in multi-core environments.
- Provides a factorial array-driven approach without inter-process communication.

Position Method: A Linear-Time Generation Algorithm for Permutations*

Dr. Yusheng Hu[a,*,1] (Independent Researcher)

[a]Independent Researcher, Shangdi East Road, Beijing, 100085, China

ARTICLE INFO

Keywords:
Permutation
Ranking and Unranking
Factorial number system
Linear Time Complexity
Parallel algorithms

ABSTRACT

This paper proposes a new permutation mapping algorithm based on the "Position Method," establishing a one-to-one correspondence between the factorial number system and permutations. Both ranking (calculating rank from permutations) and unranking (generating permutations from rank) achieve $O(n)$ time and space complexity. Experimental results demonstrate that the proposed Position Method (also referred to as the Position Pure algorithm) outperforms the Myrvold-Ruskey algorithm (Myrvold and Ruskey, 2001) in terms of constant-factor efficiency. In the iterative generation approach, the proposed algorithm is more than ten times faster than Heap's algorithm as n increases. Furthermore, the independent mapping between ranks and permutations enables highly efficient parallel algorithms with near-linear speedup and minimal synchronization overhead.

1. Introduction

A permutation refers to all ordered arrangements of n distinct elements, a topic extensively studied in existing literatureSedgewick (1977)Knuth (2005). Although algorithms such as **Heap's algorithm** Heap (1963) and **Ives's algorithm** Ives (1976) efficiently generate permutations, they are limited to generation capabilities. Algorithms with bidirectional mapping capabilities for ranking and unranking, due to their support for parallel processing, demonstrate greater practical value in engineering applications. The mapping between factorial number system and permutations is the core technical pathway for achieving this bidirectional capability. This paper proposes a specific implementation scheme Position algorithm based on the factorial number system.

Reverse Factoradic Representation The reverse factoradic system (also known as the little-endian factorial number system) is a mixed-radix representation used to encode permutations, particularly for the Myrvold-Ruskey algorithm. A non-negative integer K is uniquely represented by a sequence of digits $\mathbf{c} = (c_0, c_1, \ldots, c_{n-1})$, where the i-th digit c_i (at 0-based index i) is associated with a base of $i + 1$ and satisfies the constraint:

$$0 \le c_i \le i \quad \text{for } i = 0, 1, \ldots, n-1$$

Unlike the standard Lehmer code, the weights in this system increase from left to right. The value of K is given by:

$$K = \sum_{i=0}^{n-1} c_i \cdot (i!) = c_0 \cdot 0! + c_1 \cdot 1! + c_2 \cdot 2! + \cdots + c_{n-1} \cdot (n-1)!$$

In this notation, the sequence $(0, 0, 0, 0)$ represents the first permutation (Rank 0), and the sequence grows by incrementing the rightmost digit first, e.g., $(0, 0, 0, 1), (0, 0, 0, 2), \ldots, (0, 1, 2, 3)$, directly corresponding to the control vector required for linear-time unranking via swap operations.

Note: In this paper, permutations are ranked using the *reverse factoradic representation* (vector form), where the rank is maintained as a coefficient array $\mathbf{c} = (c_0, c_1, \ldots, c_{n-1})$ satisfying $0 \le c_i \le i$, rather than as a single large integer.

This paper presents an algorithm named **Position** with a time complexity of O(n), along with its improved variant, **Position Pure**.

*A preprint of this manuscript is available at SSRN: https://ssrn.com/abstract=6285581

*Corresponding author
dr.huyusheng@gmail.com (Y. Hu)
https://github.com/Yusheng-Hu/Position-Pure-Algorithm (Y. Hu)
ORCID(s): 0009-0005-1980-5751 (Y. Hu)

[1]This work is part of an independent research project on combinatorial algorithms.

2. Position Algorithm

The Position Method was first proposed as a novel mapping logic between the factorial number system and permutations. Building upon this foundation, the Position Pure (PP) algorithm was further developed to optimize iterative generation and ensure strict O(n) complexity Hu (2026).

2.1. Algorithm Principles

Assume that the data for permutations in the algorithm starts from 0. Let P_n denote the set of all permutations of n elements. The permutation of 1 element is $P_1 = \{0\}$.

Derivation process for permutations of 2 elements: The permutation of 1 element is {0}, denoted as P1. When extending to 2 elements, a new position (position 1) is added, which can take values (0, 1).

By appending these values to P_1, we get $\{P_1, 0\}$ and $\{P_1, 1\}$, which are expressed as follows:

$$(\mathrm{P}_1)\begin{pmatrix}0\\1\end{pmatrix} = \begin{cases}\{\mathrm{P}_1,\ 0\} = \{\{0\},\ 0\} & (1)\\ \{\mathrm{P}_1,\ 1\} = \{\{0\},\ 1\} & (2)\end{cases}$$

The {0} in (1) is 1 element permutation, because the appearance of 0 afterwards, then change {0} to the **position** , that is {1}

There is no conflict in the formula (2) , therefore keep {0,1)}

Thus, the full permutation of 2 elements is obtained

$$\begin{pmatrix}0\to 1 & 0\\ 1 & 1\end{pmatrix}\xrightarrow{After\ replace}\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}$$

Therefore, the full permutation of 3 elements is generated based on the full permutation of 2 elements:

$$\begin{pmatrix}(P_2) & 0\\ (P_2) & 1\\ (P_2) & 2\end{pmatrix}\xrightarrow{\text{assume } P_2=\begin{pmatrix}1&0\\0&1\end{pmatrix}\text{ is known}}\begin{pmatrix}\begin{pmatrix}1&0\\0&1\end{pmatrix} & 0\\ \begin{pmatrix}1&0\\0&1\end{pmatrix} & 1\\ \begin{pmatrix}1&0\\0&1\end{pmatrix} & 2\end{pmatrix}$$

Replace element in P_2 that have already appeared to 2, also as n-1,

$$\begin{pmatrix}1&0\\0&1\end{pmatrix}\begin{pmatrix}\begin{pmatrix}1 & 0\to 2\\ 0\to 2 & 1\end{pmatrix} & 0\\ \begin{pmatrix}1\to 2 & 0\\ 0 & 1\to 2\end{pmatrix} & 1\\ \begin{pmatrix}1&0\\0&1\end{pmatrix} & 2\end{pmatrix}\xrightarrow{replace}\begin{pmatrix}\begin{pmatrix}1&2\\2&1\end{pmatrix} & 0\\ \begin{pmatrix}2&0\\0&2\end{pmatrix} & 1\\ \begin{pmatrix}1&0\\0&1\end{pmatrix} & 2\end{pmatrix}\xrightarrow{final}\begin{pmatrix}1&2&0\\2&1&0\\2&0&1\\0&2&1\\1&0&2\\0&1&2\end{pmatrix}$$

the general generation method for the complete permutation P_n is

$$P_n=\begin{pmatrix}(P_{n-1}) & 0\\ (P_{n-1}) & \dots\\ (P_{n-1}) & n-1\end{pmatrix}\xrightarrow{P_{n-1}\text{ is known}}\begin{pmatrix}\begin{pmatrix}0\to n-1 & \dots & \dots\\ \dots & \dots & \dots\\ \dots & \dots & 0\to n-1\end{pmatrix} & 0\\ \begin{pmatrix}1\to n-1 & \dots & \dots\\ \dots & \dots & \dots\\ \dots & \dots & 1\to n-1\end{pmatrix} & 1\\ \dots & \dots\\ (P_{n-1}) & n-1\end{pmatrix}$$

The replacement rule for the Position algorithm is, for step i, find former value same as C[i], then replace by the position value i.

As shown in Figure 1, the replacement rule of the algorithm is illustrated.

2.2. Position algorithm and complex

The core logic of the Position algorithm is encapsulated in its ranking and unranking procedures. **Algorithm 1** details the unranking process, which transforms a factorial sequence into a permutation through a single pass by managing positional mappings. Conversely, **Algorithm 2** describes the inverse ranking procedure, which efficiently reconstructs the original factorial sequence. Both algorithms maintain an $O(n)$ time complexity and share a highly symmetrical structure, reflecting the one-to-one mapping nature of the Position Method.

Algorithm 1 Position unrank

Require: C, n
Ensure: D
1: **for** $i \leftarrow 0$ to $n-1$ **do**
2: $D[i] \Leftarrow C[i]$
3: $M[i] \Leftarrow M[C[i]]$
4: $M[C[i]] \Leftarrow i$
5: $D[M[i]] \Leftarrow i$
6: **end for**

Algorithm 2 Position rank

Require: D, n
Ensure: C
1: **for** $i \leftarrow 0$ to $n-1$ **do**
2: $C[i] \Leftarrow D[i]$
3: $M[C[i]] \Leftarrow i$
4: **end for**
5: **for** $i \leftarrow$n-1 **down to** 0 **do**
6: $C[M[i]] \Leftarrow C[i]$
7: $M[C[i]] \Leftarrow M[i]$
8: **end for**

- **Space Complexity** The algorithm uses an intermediate array $M[n]$ for computation, resulting in a space complexity of $O(n)$.
- **Time Complexity** To generate a permutation, the algorithm executes two loops each of length n. Thus, the time complexity is $O(n)$.
- **Algorithm Recursion** The complexity for generating a single permutation is $O(n)$. If permutations are output consecutively, generating the next full permutation can reuse most information from the previous one. Therefore, when generating all permutations, the complexity per permutation can be reduced to $O(1)$. The overall complexity of generating all permutations (excluding output) is $O(n!)$.

- **Parallel Algorithm** Since there is a strict one-to-one correspondence between the generation of full permutations and factoradic numbers, permutations can be generated by enumerating the factoradic numbers. The set of factoradic numbers can also be partitioned into subsets for parallel execution by specifying a start and end factoradic number for each task. This allows the task to be decomposed into parallel subtasks that can be executed across multiple devices, multiple CPU cores within a device, or on a device's GPU, thereby significantly improving the efficiency of full permutation generation. Partitioning is typically performed by fixing the first few digits of the factoradic number; specific details are omitted here.

2.3. Proof of Recursive Completeness

Assume that the permutation P_{n-1} of $n-1$ numbers is a complete permutation, i.e., it contains all permutations of the elements $\{0, 1, \dots, n-2\}$, with a total of $(n-1)!$ permutations.

When recursively generating the n-element permutation P_n according to the algorithm proposed in this paper, a new element is appended to the end of each permutation in P_{n-1}. The value of this element ranges over $\{0, 1, \dots, n-1\}$, and the specific recursive rule is:

$$P_n = P_{n-1} \cup \{x\}, \quad x \in \{0, 1, \dots, n-1\}$$

Further derivation:

1. A single complete $(n-1)$-element permutation P_{n-1} can be extended to n distinct n-element permutations.
2. Since P_{n-1} contains $(n-1)!$ permutations, the total number of n-element permutations obtained after extension is:

$$(n-1)! \times n = n!$$

3. $n!$ is exactly the total number of full permutations of the n elements $\{0, 1, \dots, n-1\}$. Therefore, the recursively generated P_n is a complete n-element permutation.

In conclusion, using the recursive rule of "appending a new element ranging from 0 to $n-1$ at the end" allows generating a complete set of n-element permutations P_n from a complete set of $(n-1)$-element permutations P_{n-1}. The recursive completeness of the algorithm is thus proven.

If the newly added digit is $n-1$, since P_{n-1} is a complete permutation set containing $(n-1)!$ distinct permutations, it directly forms $(n-1)!$ different new permutations.

When the newly added digit is among $0, \dots, n-2$, the proof proceeds as follows: Different last digits inevitably produce different permutations. What remains to be proved is that permutations ending with the same digit m do not repeat.

It is known that there are $(n-1)!$ distinct permutations preceding m. According to the algorithm, every occurrence of the value m in the set P_{n-1} is replaced by $n-1$. Since $n-1$ does not appear in P_{n-1}, replacing m cannot cause a duplication of permutations. Therefore, after applying the algorithm, all permutations ending with suffix m retain the same number as P_{n-1}, i.e., $(n-1)!$ distinct permutations.

Since the last digit can take values from 0 to $n-1$, that's n possibilities in total. Hence, the total number of permutations in the new set P_n is n times P_{n-1}. i.e., $n \cdot (n-1)! = n!$. Proven.

3. Position Pure Algorithm

Acceleration Approach It is observed that the $(n-1)$-th position needs to be swapped with the digits at positions $0, \dots, n-2$. The swapping rule is applied when the digit at the $(n-1)$-th position already appears among the preceding digits. Since the digit $n-1$ itself occurs at position $n-1$, no swap is required at that stage. Ignoring the exact positions and considering sequential swapping in the order $0, \dots, n-1$, we have:

Assume an existing $(n-1)$-permutation: $D[0]D[1]\dots D[n-2]$. When extending it by appending $D[n-1]$ (where $D[n-1] \in \{0, \dots, n-1\}$), the procedure is as follows. The possible exchange processes are listed in **Table 1**

Acceleration Strategy Therefore, an acceleration strategy can be employed in the process: From the 0-th position up to the $(n-2)$-th position, assign the value $n-1$ to each corresponding location in turn and change the element originally at that location to $n-1$. That is, first set $D[n-1] = n-1$, which itself constitutes a permutation; then successively interchange $D[i]$ (for $i = 0, \dots, n-2$) with $D[n-1]$ to produce $n-2$ additional permutations. In this way a total of $n-1$ permutations are generated directly in a loop, without needing to explicitly check any positions.

Table 1
All possible exchange processes for the (n-1)th position

first n-2 position	last position	process procedure
[n-1]$D[1]\dots D[n-2]$	$D[0]$	$D[n-1] \leftarrow D[0],\ D[0] \leftarrow [n-1]$
$D[0]$ **[n-1]**$\dots D[n-2]$	$D[1]$	$D[n-1] \leftarrow D[1],\ D[1] \leftarrow [n-1]$
…	…	…
$D[0]D[1]\dots$ **[n-1]**	$D[n-2]$	$D[n-1] \leftarrow D[n-2],\ D[n-2] \leftarrow [n-1]$
$D[0]D[1]\dots D[n-2]$	$[n-1]$	$D[n-1] \leftarrow [n-1]$

New Mapping Insights from the Acceleration Approach The previous analysis reveals that the newly introduced element $n-1$ can occupy exactly $n-1$ possible positions when being inserted into the permutation. This coincides precisely with the range of values for the $(n-1)$-th digit of the factorial-number storage (denoted as C). Leveraging this correspondence, a mapping algorithm can be developed: at each insertion step, $k \in \{0, 1, \dots, n-1\}$ chooses a position according to digit $C[k]$.

Assuming an $(n-2)$-order permutation $D[0]\dots D[n-2]$ is given, to obtain an $(n-1)$-order permutation with $n-1$ inserted, we set $k = n-1$ and do: take $D[C[n-1]]$ and assign it to $D[n-1]$, then set $D[C[n-1]] \leftarrow n-1$.

This algorithm is both concise and elegant: each round of the loop selects an element from the existing permutation to be swapped out, and places k into the swapped position. Thus each permutation can be obtained by performing exactly $(n-1)$ swaps starting with $[0, 1, \dots, n-1]$. Its time complexity is clearly $O(n)$, which maintains high efficiency.

algorithm: for the step i, C[i]=m, assign the m-th position C[m] value to C[i], then C[m] set to i.

As shown in Figure 2, the replacement rule of the algorithm is illustrated.

The optimized **Position Pure algorithm** further refines the mapping process, achieving even greater structural simplicity. As shown in **Algorithm 3**, the unranking operation is reduced to a minimal iterative swap-like logic, which significantly lowers the constant-factor overhead during permutation generation. Correspondingly, **Algorithm 4** provides the inverse ranking procedure, maintaining the same $O(n)$ efficiency while ensuring a precise one-to-one mapping. These streamlined procedures are the key to the algorithm's superior performance in iterative and parallel computing tasks.

Algorithm 3 PositionPure unrank

Require: C, n
Ensure: D
1: **for** $i \leftarrow 0$ to $n-1$ **do**
2: $\quad D[i] \Leftarrow D[C[i]]$
3: $\quad D[C[i]] \Leftarrow i$
4: **end for**

Algorithm 4 PositionPure rank

Require: D, n
Ensure: C
1: **for** $i \leftarrow 0$ to $n-1$ **do**
2: $\quad M[D[i]] \Leftarrow i$
3: **end for**
4: **for** $i \leftarrow$n-1 **down to** 0 **do**
5: $\quad C[i]] \Leftarrow M[i]$;
6: $\quad D[M[i]] \Leftarrow D[i]$;
7: $\quad M[D[i]] \Leftarrow M[i]$;
8: **end for**

4. Summary

This paper presents the Position Method and its optimized variant, the Position Pure algorithm, for establishing a direct mapping between the factorial number system and permutations. Both algorithms achieve an optimal time and space complexity of $O(n)$, providing a more efficient constant-factor performance compared to the traditional Myrvold-Ruskey algorithm.

Experimental evaluations demonstrate that the Position Pure algorithm is more than ten times faster than the classic Heap's algorithm during iterative generation as n increases. Furthermore, the inherent independence of the mapping logic enables highly efficient **parallel** generation with near-linear speedup and minimal synchronization overhead. The conciseness and lack of recursive redundancy make these algorithms particularly suitable for high-performance computing, random sampling, and large-scale permutation tasks.

Table 2
Example: 4-digit number unranking process

position	read	pending	conflict	identification	correction	result
0	C[0] = 0	0	None	0	None	0
1	C[1] = 1	01	None	01	None	01
2	C[2] = 1	011	C[1] = 1	$0\underline{1}1$	$0\langle 1\rangle 21$	021
3	C[3] = 2	0212	C[1] = 2	$0\underline{2}12$	$0\langle 2\rangle 312$	0312

Table 3
Permutation mapping of 4 elements (Position unrank)

Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$	Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$	Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$
0	0 0 0 0	1 2 3 0	8	0 0 2 0	1 3 2 0	16	0 1 1 0	3 2 1 0
1	0 0 0 1	3 2 0 1	9	0 0 2 1	3 0 2 1	17	0 1 1 1	0 2 3 1
2	0 0 0 2	1 3 0 2	10	0 0 2 2	1 0 3 2	18	0 1 1 2	0 3 1 2
3	0 0 0 3	1 2 0 3	11	0 0 2 3	1 0 2 3	19	0 1 1 3	0 2 1 3
4	0 0 1 0	2 3 1 0	12	0 1 0 0	2 1 3 0	20	0 1 2 0	3 1 2 0
5	0 0 1 1	2 0 3 1	13	0 1 0 1	2 3 0 1	21	0 1 2 1	0 3 2 1
6	0 0 1 2	3 0 1 2	14	0 1 0 2	3 1 0 2	22	0 1 2 2	0 1 3 2
7	0 0 1 3	2 0 1 3	15	0 1 0 3	2 1 0 3	23	0 1 2 3	0 1 2 3

A. Position algorithm procedure

unranking procedure Referring to the algorithm described above, for n-carry arrays, only one traversal is needed to generate the corresponding full permutation. **Table 2** illustrates this step-by-step unranking process for a 4-digit number, demonstrating how the algorithm identifies element conflicts and applies corrections to ensure a valid permutation in linear time.

Taking the 4-digit number 0112 as an example, Traverse (according to sequence numbers 0-3):

- Check the 0th position value is 0, There is no number in front, no change. get 0
- Check the 1st position value is 1, There is no other 1 in front, no change. get 01
- Check the 2nd position value is 1, There is 1 in position 1, change to position value 2. get $02(1 \to 2)1$
- Check the 3rd position value is 2, There is 2 in position 1, change to position value 3. get $03(2 \to 3)12$

ranking procedure According to the Position algorithm, as long as the operation is reversed, a reverse Map mapping can be performed. Traverse the variable i from $n-1$ to 0, first traversing the array to create a positional index. Then traverse the variable i from $n-1$ to 0, judgment: If there is a value $D[i]$ in the D array that is the same as current position i, then this value is rewritten by repeating the number above base and needs to be restored to $D[i]$. Change $D[i]$ to i.

The complete mapping results between decimal values (Dec), factorial number sequences ($C_0C_1C_2C_3$), and the generated permutations ($D_0D_1D_2D_3$) for 4 elements are summarized in **Table 3**. This table exemplifies the one-to-one correspondence achieved by the Position unranking process across the entire permutation space.

Table 4
Permutation mapping of 4 elements (Position Pure unrank)

Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$	Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$	Dec	$C_0C_1C_2C_3$	$D_0D_1D_2D_3$
0	0 0 0 0	3 0 1 2	8	0 0 2 0	3 0 2 1	16	0 1 1 0	3 2 1 0
1	0 0 0 1	2 3 1 0	9	0 0 2 1	1 3 2 0	17	0 1 1 1	0 3 1 2
2	0 0 0 2	2 0 3 1	10	0 0 2 2	1 0 3 2	18	0 1 1 2	0 2 3 1
3	0 0 0 3	2 0 1 3	11	0 0 2 3	1 0 2 3	19	0 1 1 3	0 2 1 3
4	0 0 1 0	3 2 0 1	12	0 1 0 0	3 1 0 2	20	0 1 2 0	3 1 2 0
5	0 0 1 1	1 3 0 2	13	0 1 0 1	2 3 0 1	21	0 1 2 1	0 3 2 1
6	0 0 1 2	1 2 3 0	14	0 1 0 2	2 1 3 0	22	0 1 2 2	0 1 3 2
7	0 0 1 3	1 2 0 3	15	0 1 0 3	2 1 0 3	23	0 1 2 3	0 1 2 3

The mapping results of the **Position Pure algorithm** are highly intuitive and maintain a strict one-to-one correspondence across the entire factorial space. **Table 4** detailed the specific transformations from decimal indices (Dec) and factorial sequences ($C_0C_1C_2C_3$) to the final permutations ($D_0D_1D_2D_3$) for $n = 4$. Notably, while the underlying $O(n)$ complexity remains consistent with the original Position algorithm, the refined mapping logic in the Pure version produces a distinct and more efficient distribution of permutations.

Table 5
Performance comparison between Heap's algorithm and Position Pure algorithm (Environment: AMD EPYC 7763 64-Core)

n	Heap (s)	Position Pure (s)	Speedup
9	0.005729	0.000998	5.74×
10	0.058123	0.006297	9.23×
11	0.645687	0.064628	9.99×
12	7.834273	0.720868	10.86×
13	103.662456	8.662150	11.96×

Table 6
Parallel Performance on Dual Independent CPU Cores (Environment: AMD EPYC 7763 64-Core)

N	Core 0 Time (s)	Core 1 Time (s)	Core 0 Thr. (G/s)	Core 1 Thr. (G/s)
10	0.0029	0.0029	0.63	0.63
11	0.0310	0.0308	0.64	0.65
12	0.3597	0.3585	0.67	0.67
13	4.5065	4.5037	0.69	0.69

Table 7
Performance comparison of linear-time ranking/unranking: Position Pure (PP) vs. Myrvold-Ruskey (MR) (Random distribution)

N	Myrvold-Ruskey (ns/it)	Position Pure (ns/it)	Speedup
1,000	1057.0	633.4	1.67×
100,000	114563.3	94561.8	1.21×
1,000,000	1568522.4	1247111.3	1.26×

B. Algorithm Compare

B.1. Full Permutation Generation Performance

Comparative Analysis with Heap's Algorithm: For decades, Heap's algorithm has been the gold standard for generating all permutations, widely regarded as the most efficient implementation due to its minimal element movements. In this study, we compared the execution time of the Position Pure algorithm with Heap's algorithm using GitHub Actions runners. As shown in Table 5, despite the long-standing dominance of Heap's method, the Position Pure algorithm significantly outperforms it, achieving a speedup of approximately 11.96× when n=13.

B.2. Parallel Generation Efficiency

Parallel Efficiency: To evaluate the scalability of the Position Pure algorithm, we implemented a parallel version using two independent CPU cores on GitHub Actions. The experimental data summarized in Table 6 indicates that the total execution time remains consistent with the single-core performance. This demonstrates that the algorithm possesses excellent parallel capability with near-linear speedup, as there are no sequential dependencies between permutations.

B.3. Linear-time Unranking and Ranking Analysis

Myrvold-Ruskey Algorithm Myrvold and Ruskey (2001): A breakthrough algorithm proposed in 2001 that solved the linear-time Unranking problem. Unlike traditional methods, it utilizes a specific non-lexicographical ordering to achieve Rank and Unrank operations in strict $O(n)$ time without complex data structures. Parks and Wills (2021) proposed $O(n)$ functions via DTR, clarifying the algorithm with an algebraically meaningful order. As demonstrated in Table 7, our proposed Position Pure algorithm maintains the same $O(n)$ complexity while achieving a performance improvement of approximately 1.21× to 1.67× compared to the Myrvold-Ruskey algorithm.

Note: The metric ns/it refers to nanoseconds per iteration, representing the average time required to complete one Rank or Unrank operation.

CRediT authorship contribution statement

Yusheng Hu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - Original draft, Visualization.

References

Heap, B.R., 1963. Permutations by interchanges. The Computer Journal 6, 293–294.

Hu, Y., 2026. Position pure algorithm: A linear-time generation algorithm for permutations. Zenodo. V1.3.0, https://doi.org/10.5281/zenodo.18728595.

Ives, F., 1976. Permutation enumeration: Four new permutation algorithms. Communications of the ACM 19, 68–72.

Knuth, D.E., 2005. The Art of Computer Programming, Volume 4, Fascicle 2: Generating All Tuples and Permutations. Addison-Wesley Professional, Reading, MA.

Myrvold, W., Ruskey, F., 2001. Ranking and unranking permutations in linear time. Information Processing Letters 79, 281–284.

Sedgewick, R., 1977. Permutation generation methods. Computing Surveys 9, 137–164.

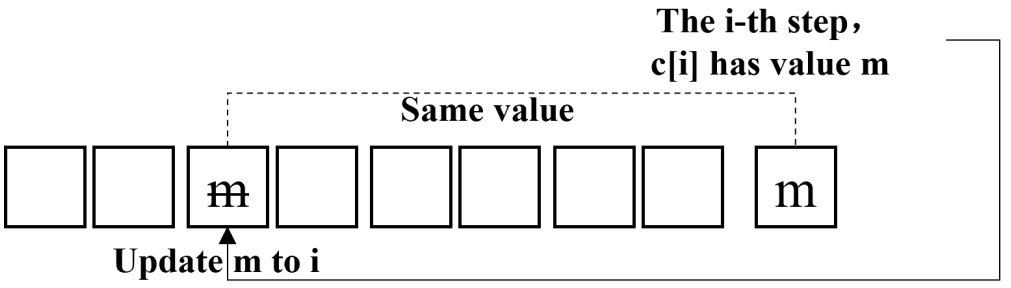

Figure 1: Replacement rule for the Position algorithm

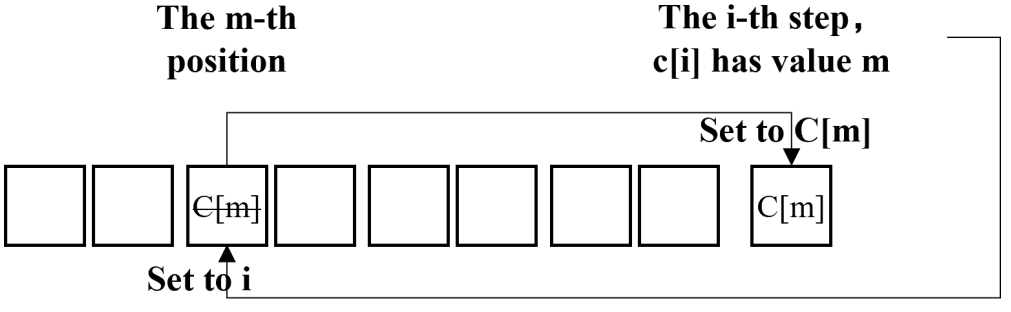

Figure 2: Replacement rule for the Position Pure algorithm